agentmemory 从入门到精通

项目	说明
教程对象	agentmemory（npm 包名 `@agentmemory/agentmemory`），AI 编码代理的持久记忆引擎
基于版本	v0.9.30（内置 iii 引擎固定为 v0.22.1）
核实日期	2026 年 10 月 24 日
依据来源	官方文档站 agent-memory.dev/docs、GitHub 仓库 rohitg00/agentmemory 的 README 与配置文档、npm 发布信息
许可证	Apache-2.0
运行环境假设	命令行示例以 macOS 与 Linux 的 bash 为主，Windows 差异在各章单独注明；读者需具备基本的终端操作能力，不要求 Node.js 开发经验

agentmemory 迭代较快，本文全部命令、参数与行为均按上述版本核实。若使用更新的版本，个别行为可能与本文不同，届时以官方文档（https://agent-memory.dev/docs）为准。

术语约定：本文首次出现的专有名词均给出定义，此后统一使用。常见术语如下。

术语	英文	含义
记忆引擎	memory engine	负责记录、压缩、检索与遗忘代理工作过程的本地服务
MCP	Model Context Protocol	模型上下文协议，一种让 AI 客户端调用外部工具的标准协议
钩子	hook	编码代理在会话或工具调用等生命周期节点自动执行的外部脚本
观测	observation	钩子捕获的原始事件，包括提示词、工具调用及其结果
检索	recall	按查询条件从记忆库中找出相关记录并排序返回的过程

第一部分 入门篇：从零到第一次记忆

本部分回答三个问题：agentmemory 解决什么问题、如何在本地把它运行起来、如何让编码代理真正用上记忆并验证记忆闭环确实生效。完成本部分后，读者将拥有一个正在运行的记忆服务、一个已接入的编码代理，以及一次可复现的跨会话记忆验证。

第 1 章 认识 agentmemory：编码代理的记忆难题与解决思路

1.1 编码代理的记忆问题

使用 Claude Code、Cursor、Codex CLI 这类 AI 编码代理时，每个新会话都从零开始：项目的技术选型需要重新解释，上次调试通过的方案需要重新发现，个人偏好需要反复说明。各代理自带的记忆机制（如 CLAUDE.md、Cursor 的 notepad、Cline 的 memory bank）本质上是手工维护的静态文件，存在三个限制：容量有限（CLAUDE.md 的有效加载范围约为前 200 行）、内容随项目演进而过期、且不能在不同代理之间共享。

1.2 agentmemory 是什么

agentmemory 是一个面向 AI 编码代理的持久记忆引擎（memory engine，负责记录、压缩、检索与遗忘工作过程的本地服务），同时也是一个 MCP 服务器（MCP 即 Model Context Protocol，模型上下文协议，是 AI 客户端调用外部工具的标准协议）。它驻留在后台，通过代理的钩子（hook，代理在会话与工具调用等生命周期节点自动执行的脚本）静默捕获每一次工具调用、文件编辑与决策，将其压缩为可检索的结构化记忆，并在下一次会话开始时注入恰当的上下文。

官方对它的定位描述为：为 Claude Code、GitHub Copilot CLI、Cursor、Gemini CLI、Codex CLI、Hermes、OpenClaw、pi、OpenCode 以及任何 MCP 客户端提供持久记忆。所有接入的代理共享同一个本地记忆服务器。

该项目的几个基本事实（核实于 2026 年 10 月 24 日）：

- GitHub 星标约 2.98 万，npm 累计安装量约 50.1 万；
- 本地优先：数据保存在本机，无账户、无云端依赖、默认无外部数据库；
- 无密钥可用：捕获、索引与关键词检索不需要任何 LLM 密钥；配置密钥后解锁摘要、反思、巩固等高级能力；
- 内部构建在 iii 引擎（iii engine，一个以 worker、function、trigger 三原语组合的分布式后端运行时）之上，固定使用 v0.22.1 版本。

1.3 与内置记忆方案的区别

官方 README 将内置记忆比作便签，将 agentmemory 比作便签背后的可检索数据库。两者的对比如下。

维度	内置方案（如 CLAUDE.md）	agentmemory
容量	约 200 行上限	无硬性上限
检索方式	全量载入上下文	BM25、向量与图谱三路融合，仅取前若干条
上下文成本	约 240 条观测时超过 22 千 token	每会话约 1900 token（减少约 92%）
跨代理	每个代理各自维护文件	通过 MCP 与 REST 被所有代理共享
多代理协同	无	支持租约、信号、actions、例程
可观测性	手工阅读文件	3113 端口实时查看器

表中 token 指大语言模型处理文本的计量单位；BM25 是一种基于词频的经典关键词检索算法，第 6 章将详细解释。

1.4 核心能力总览

能力	说明
自动捕获	通过钩子记录每次工具调用，无需人工干预
混合检索	BM25、向量与知识图谱三路信号，经倒数排名融合排序
记忆演化	版本链、新版本取代旧版本、关系图谱
自动遗忘	过期清除、矛盾检测、按重要性淘汰
隐私过滤	存储前剥离 API 密钥、机密信息与 `<private>` 标签内容
自我修复	断路器、提供方回退链、健康监控
Claude 桥接	可读取 MEMORY.md 并导出项目记忆，不覆盖既有文件
知识图谱	实体抽取与图遍历
团队记忆	按命名空间在成员之间共享或隔离
溯源引用	任何记忆都可回溯到来源观测
状态快照	记忆状态的版本化、回滚与差异对比
实时查看器	3113 端口的可视化面板与会话重放

1.5 架构总览：一个进程，三个组成部分

单个 agentmemory 进程承载三个部分：内置 iii 引擎（提供状态存储、事件流与 HTTP 触发器）、记忆工作进程（向引擎注册全部 `mem::*` 系列函数，共 311 个）、以及查看器。代理集成通过 REST 接口或 MCP 服务器交互，两者最终分派到同一组工作进程函数。

运行时占用四个端口：

端口	用途
3111	REST API 与 MCP HTTP 接口
3112	iii 事件流
3113	实时查看器
49134	iii 引擎工作进程 WebSocket（内部使用）

若修改 REST 端口，则事件流为 N+1、查看器为 N+2、引擎为 N+46023，四个端口整体联动。

1.6 适用与不适用场景

适用场景：个人开发者长期使用一个或多个编码代理、希望在项目间保留架构决策与调试经验、团队需要共享代理记忆、需要审计代理行为的合规场景。

需要斟酌的场景：对数据必须完全出网有强合规要求的团队应评估团队共享与 mesh 同步功能的网络行为；只需单一代理且记忆量极小的用户，内置的 CLAUDE.md 可能已经足够；不愿运行常驻后台进程的环境需要权衡资源占用。

第 2 章 安装与首次运行

2.1 环境要求

项目	要求
Node.js	20 或更新版本，需附带 npm 与 npx（用 `node -v`、`npm -v`、`npx -v` 确认）
macOS 与 Linux 自动安装引擎	需要 curl、POSIX sh 与 tar；`node:20-slim` 等精简镜像可能缺少这三项
原生 Windows	需手动安装固定版本 iii 引擎的 `iii.exe`，或改用 WSL2、Docker Desktop

2.2 标准安装与首次运行

官方给出的标准安装命令如下：

```
npx -y @agentmemory/agentmemory@latest
```

参数含义：`npx` 从 npm 临时下载并运行包；`-y` 自动确认 npx 的安装提示；`@latest` 显式取最新版本，避免命中本地缓存的旧版本。

首次运行是交互式设置流程，依次完成：选择要接入的编码代理（Claude Code、Cursor、Codex、Gemini CLI、OpenCode 等）、选择 LLM 提供方或保持无密钥模式、写入配置、启动记忆服务与其固定的 iii 引擎，并提示是否全局安装以便在任何目录直接使用 `agentmemory` 命令。

执行成功后，终端会显示启动面板，列出四个端口的服务状态。首次运行还会在 `~/.agentmemory/secret` 生成访问密钥（权限 0600），细节见第 4.3 节。

全局安装是另一种方式，适合长期使用：

```
npm install -g @agentmemory/agentmemory
agentmemory
```

两种方式的对比如下。

方式	优点	注意事项
`npx -y @agentmemory/agentmemory@latest`	无需预装，始终可取最新版	npx 按版本缓存，裸写 `npx @agentmemory/agentmemory` 可能命中旧缓存
`npm install -g`	命令随处可用，无缓存歧义	升级需手动 `npm update -g`

从源码运行的方式见第 2.4 节。

2.3 Windows 安装

agentmemory 可运行于 Windows 10 与 11，但仅有 Node.js 包不够，还需要固定版本（v0.22.1）的 iii 引擎作为后台进程。CLI 不会自动解包 Windows 压缩包，原生 Windows 用户有三条路径。

方式一：手动安装预编译引擎二进制（推荐）。

```
# 1. 浏览器打开 https://github.com/iii-hq/iii/releases/tag/iii%2Fv0.22.1
# 2. 下载 iii-x86_64-pc-windows-msvc.zip (ARM 机器选 iii-aarch64-pc-windows-msvc.zip)
# 3. 将其中的 iii.exe 放入 agentmemory 的引擎目录:
New-Item -ItemType Directory -Force "$HOME\.agentmemory\bin"
# 将 iii.exe 复制到 $HOME\.agentmemory\bin\iii.exe
# 4. 验证版本，应输出 0.22.1:
& "$HOME\.agentmemory\bin\iii.exe" --version
# 5. 正常启动 agentmemory:
npx -y @agentmemory/agentmemory@latest
```

方式二：Docker Desktop。安装并启动 Docker Desktop 后执行：

```
$env:AGENTMEMORY_USE_DOCKER = "1"
npx -y @agentmemory/agentmemory@latest
```

方式三：仅独立 MCP 服务器。若只需要 MCP 工具而不需要 REST API、查看器与定时任务，可完全跳过引擎：

```
npx -y @agentmemory/agentmemory@latest mcp
# 或使用独立的 shim 包
npx -y @agentmemory/mcp
```

Windows 启动失败的常见症状与处理：

症状	处理
引擎进程已启动但 REST API 无响应	确认四个端口空闲、`iii.exe` 仍在运行，然后加 `--verbose` 重跑并查看引擎错误输出
提示无法启动 iii-engine	未安装 `iii.exe` 且未启用 Docker，按方式一或方式二处理
端口冲突	用 `netstat -ano
已装 Docker 但回退未生效	确认 Docker Desktop 正在运行（系统托盘图标）

2.4 从源码运行

```
git clone https://github.com/rohitg00/agentmemory.git && cd agentmemory
npm install && npm run build && npm start
```

该方式在固定引擎二进制已安装时使用本地引擎，否则可选择 Docker Compose。REST、事件流与查看器默认绑定 127.0.0.1。macOS 与 Linux 的引擎自动安装需要 curl、POSIX sh 与 tar。

如需手动安装引擎，agentmemory 固定使用 v0.22.1（与其依赖的 iii-sdk 同版本，二者必须配套）。以 macOS ARM 为例：

```
mkdir -p ~/.local/bin && curl -fsSLo iii.tar.gz \
  https://github.com/iii-hq/iii/releases/download/iii/v0.22.1/iii-aarch64-apple-darwin.tar.gz \
  && echo "2b309019b909a896cae874dc947e2cdf877b4f3c51dd026b79850af858517fa4  iii.tar.gz" \
  | shasum -a 256 -c - \
  && tar -xzf iii.tar.gz -C ~/.local/bin && chmod +x ~/.local/bin/iii
```

其余平台替换压缩包文件名与校验值即可，发布页每个压缩包都有对应的 `.sha256` 文件；Windows 使用 `Get-FileHash` 校验。注意 iii 引擎是预编译二进制，不能用 `cargo install` 安装；上游 `install.sh` 脚本安装的是最新版引擎，agentmemory 不支持，必须使用上述固定版本的安装方式。

2.5 数据目录与端口规划

默认情况下，引擎状态存储在仓库之外的平台目录，避免污染项目目录：

平台	默认数据目录
macOS	`~/Library/Application Support/agentmemory`
Linux	`$XDG_DATA_HOME/agentmemory`，未设置时为 `~/.local/share/agentmemory`
Windows	`%APPDATA%\agentmemory`

配置与密钥文件则位于 `~/.agentmemory/`：`.env` 存放环境变量（第 9 章），`secret` 存放自动生成的访问密钥（第 4.3 节），`bin/` 存放引擎二进制，`backups/` 存放 connect 命令的配置备份。

两种方式可显式指定数据目录，且显式设置的优先级高于历史遗留的发现逻辑：

```
npx -y @agentmemory/agentmemory@latest --data-dir ~/.agentmemory-projects/main
AGENTMEMORY_DATA_DIR=~/.agentmemory-projects/main npx -y @agentmemory/agentmemory@latest
```

参数说明：`--data-dir` 为命令行覆盖；`AGENTMEMORY_DATA_DIR` 为环境变量回退，仅在未传命令行参数时生效。自定义目录后，每次重启都必须传入相同值，否则会读到另一个空目录。

多实例并行使用 `--instance <N>`（N 最大 50），等价于 REST 端口取 3111+N×100：`--instance 1` 对应 3211/3212/3213/49234 四端口，数据目录自动追加 `instance-1` 后缀。

2.6 首次运行后的系统状态

首次运行完成后，系统处于如下状态：一个 agentmemory 进程运行中，内含引擎、工作进程与查看器；`~/.agentmemory/.env` 已按交互选择生成；`~/.agentmemory/secret` 已生成访问密钥；3111、3112、3113、49134 四个端口已被占用；浏览 `http://localhost:3113` 可以打开查看器（此时记忆为空属正常现象）。

第 3 章 连接第一个 agent：connect 命令与 Claude Code 插件

本章解决的问题是让编码代理与记忆服务真正连通。代理接入有三条通道：钩子（自动捕获每个事件）、MCP（代理按需调用记忆工具）、REST（任何能发 HTTP 请求的程序直接调用）。三条通道可以叠加使用。

3.1 方式一：connect 命令

connect 是官方推荐的接入方式，支持 21 个代理适配器。不带参数运行会打开交互式选择器，列出本机检测到的代理：

```
npx -y @agentmemory/agentmemory@latest connect
```

每个自动化适配器在写入前会把目标配置文件备份到 `~/.agentmemory/backups/`，写入后重新读取文件验证条目确实落盘。常用参数如下：

参数	作用
`--all`	一次性接入所有检测到的代理
`--dry-run`	只打印将要做的修改，不改任何文件
`--force`	在已有 agentmemory 条目时强制重装
`--with-hooks`	在代理支持时额外安装原生自动捕获钩子，涉及 Claude Code（`~/.claude/settings.json`）、Codex（`~/.codex/hooks.json`）、Droid（`~/.factory/hooks.json`）、Antigravity（共享 `~/.gemini/config/hooks.json`）
`--no-guidelines`	跳过写入代理规则文件的记忆使用指引

需要注意三点。第一，Windows 上自动化 connect 仅支持 `copilot-cli`，其余代理需按第 3.3 节手工配置。第二，钩子安装引用安装包内 `plugin/` 目录的绝对路径，升级 agentmemory 后需重跑 `connect <agent> --with-hooks` 刷新路径。第三，connect 写入的 MCP 配置指向 `http://localhost:3111` 且不内嵌密钥，`@agentmemory/mcp` 会在启动时自行解析密钥（环境变量、`~/.agentmemory/.env`、生成的 `~/.agentmemory/secret` 三级顺序），且仅发送给本机回环地址。

以接入 Cursor 为例：

```
agentmemory connect cursor
```

执行后命令行会打印写入的文件与验证结果，随后重启 Cursor 或开启新会话即可生效。

3.2 方式二：Claude Code 原生插件（推荐路径）

对 Claude Code，官方推荐插件方式，一次性完成全部注册。先在独立终端启动记忆服务，再在 Claude Code 内执行两条命令：

```
/plugin marketplace add rohitg00/agentmemory
/plugin install agentmemory
```

插件注册三样东西：全部 12 个钩子、17 个技能（skill，供代理按需调用的指令包）、以及通过插件自带 `.mcp.json` 自动接入的 `@agentmemory/mcp` MCP 服务器，从而无需额外配置即可使用全部 54 个 MCP 工具（如 `memory_smart_search`、`memory_save`、`memory_governance_delete`）。

存在一个已知的替代路径问题：若不装插件而是直接在 `~/.claude.json` 手工写 MCP 配置，Claude Code 不会解析 `${CLAUDE_PLUGIN_ROOT}` 变量，钩子脚本必须写成绝对路径，而这些路径通常内嵌了版本号，下次升级后所有钩子会静默失效。官方给出的补救方式是：

```
agentmemory connect claude-code --with-hooks
```

该命令把同样的钩子命令以绝对路径合并进 `~/.claude/settings.json`，并保留文件中与 agentmemory 无关的既有条目；升级后需重跑一次以刷新路径。远程或有访问保护的部署中，启动 Claude Code 前设置 `AGENTMEMORY_URL` 与 `AGENTMEMORY_SECRET` 环境变量即可。

3.3 方式三：手工 MCP 配置块

对于任何使用 `mcpServers` 配置结构的代理（Cursor、Claude Desktop、Cline、Roo Code、Gemini CLI 等），接入的是同一个标准块：

```
{
  "mcpServers": {
    "agentmemory": {
      "command": "npx",
      "args": ["-y", "@agentmemory/mcp"],
      "env": {
        "AGENTMEMORY_URL": "${AGENTMEMORY_URL}",
        "AGENTMEMORY_SECRET": "${AGENTMEMORY_SECRET}"
      }
    }
  }
}
```

操作要点：将该条目合并进宿主配置文件已有的 `mcpServers` 对象中，不要整文件替换；若文件尚无 `mcpServers` 键，则在根对象内新增。`${VAR}` 占位符在 MCP 服务器启动时从环境继承，未设置时传空字符串，shim 会回退到 `http://localhost:3111`。

OpenCode 的配置结构不同，使用顶层 `mcp` 键且命令为数组：

```
{
  "mcp": {
    "agentmemory": {
      "type": "local",
      "command": ["npx", "-y", "@agentmemory/mcp"],
      "enabled": true
    }
  }
}
```

常见代理的配置文件位置节选如下，完整列表见官方 Connectors 文档。

代理	配置文件
Cursor	`~/.cursor/mcp.json`
Claude Desktop	`claude_desktop_config.json`（应用支持目录）
Gemini CLI	`~/.gemini/settings.json`（可用 `gemini mcp add agentmemory npx -y @agentmemory/mcp --scope user` 自动合并）
Codex CLI	`~/.codex/config.toml`（TOML 形式 `[mcp_servers.agentmemory]`）
Cline	`~/.cline/mcp.json` 或设置界面
Zed	`~/.config/zed/settings.json`（注意 Zed 使用 `context_servers` 键）

修改后重启对应代理生效；Claude Code 与 Copilot 也可在会话内执行 `/mcp` 重新加载。

3.4 安装技能：让代理知道何时使用记忆

接入（connect）让工具可用，技能（skills）让代理知道何时该用。安装官方的 17 个技能：

```
npx skills add rohitg00/agentmemory -y
```

skills CLI 会自动检测发起调用的代理并写入其原生技能目录，支持 50 余个代理；`-a warp` 显式指定代理，`-a '*'` 安装到所有已安装代理。该命令与 connect 互补，建议两者都执行。对技能 CLI 尚未覆盖的代理（如 Zed 1.3.x 及更早版本），可手动将技能文件复制到代理的原生技能目录。

3.5 重启与生效验证

接入完成后执行：重启代理或开启新会话（钩子与 MCP 服务器只在会话启动时加载）。在已接入的代理里进行一次普通编码对话（例如让代理读取一个文件并修改），然后打开 `http://localhost:3113` 查看器的 Sessions 页，能看到观测条目陆续出现，即说明捕获链路已通。第 4 章给出更完整的检索侧验证。

第 4 章 验证记忆闭环：demo、检索与实时查看器

本章的目标是完成一次可复现的端到端验证：写入一条记忆、检索到它、重启服务后仍然检索到它。完成本章后，读者可以确信安装、捕获、索引与检索四环节全部正常。

4.1 用 demo 建立样例记忆

官方提供 demo 子命令，播种三段贴近真实的会话样例（JWT 认证实现、N+1 查询修复、速率限制）并对其执行检索：

```
npx -y @agentmemory/agentmemory@latest demo
```

预期输出为几组检索结果，关键词查询（如认证、中间件）应当命中。`--serve` 参数可在一个命令内完成启动服务、运行演示、停止服务的全过程：

```
npx -y @agentmemory/agentmemory@latest demo --serve
```

无密钥模式下向量被禁用，demo 中语义化的查询 `database performance optimization` 可能返回零结果，这属于预期行为，处理方式见第 6.4 节。

4.2 健康检查与状态确认

服务运行时，依次执行以下检查：

```
curl -fsS http://localhost:3111/agentmemory/livez
curl -fsS http://localhost:3111/agentmemory/health
curl -fsS -o /dev/null http://localhost:3113/
npx -y @agentmemory/agentmemory@latest status
```

命令说明：`/agentmemory/livez` 为存活探针，正常返回 200；`/agentmemory/health` 返回各子系统健康详情；第三个命令确认查看器页面可访问；`status` 子命令打印连接状态、记忆数量、启用的特性标志与健康报告。示例中 curl 的参数含义：`-f` 使 HTTP 错误返回非零退出码，`-s` 关闭进度显示，`-S` 在静默模式下仍打印错误，`-o /dev/null` 丢弃响应体。

4.3 手工写入与检索探针

REST API 默认要求持有者令牌（bearer token）认证。首次启动时，服务自动在 `~/.agentmemory/secret` 生成随机密钥（文件权限 0600），CLI、钩子、查看器与 MCP 客户端都会自动读取它，因此只有手工构造的请求需要显式携带 `Authorization` 头。以下探针先写入一条记忆：

```
curl -fsS -X POST http://localhost:3111/agentmemory/remember \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $(cat ~/.agentmemory/secret)" \
  -d '{"content":"agentmemory restart persistence probe","concepts":["install-check"]}'
```

参数说明：`remember` 端点保存一条显式记忆；`content` 为记忆正文；`concepts` 为概念标签数组，用于后续检索匹配。预期输出为包含新记忆标识符的 JSON 对象。

再执行检索：

```
curl -fsS -X POST http://localhost:3111/agentmemory/smart-search \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $(cat ~/.agentmemory/secret)" \
  -d '{"query":"restart persistence probe","limit":5}'
```

预期输出为一个 JSON 结果列表，且探针记忆应出现在结果中。`limit` 限制返回条数。看到结果即说明捕获、索引与混合检索全部工作正常。

4.4 实时查看器

查看器随服务自动启动，绑定 127.0.0.1。它连接时加载一次快照，此后通过事件流实时更新，无需轮询。界面共 12 个标签页，分四组：Memories（记忆检索、按项目与类型过滤、版本链详情、编辑与遗忘）、Sessions（会话内观测时间线）、Graph（知识图谱）、Health（健康报告，每个问题附带修复建议），以及 Audit、Activity、Profile、Replay、Lessons、Actions、Crystals 等页面。

Replay 页值得单独说明：每个被记录的会话都可重放，可按时间轴拖动查看提示词、工具调用、工具结果与响应，支持 0.5 到 4 倍速播放，空格键切换播放、方向键逐条步进。

```
open http://localhost:3113
```

4.5 重启持久化验证

持久化是记忆系统的核心承诺，官方建议显式验证一次：

```
npx -y @agentmemory/agentmemory@latest stop
```

`stop` 会先等待一个状态落盘窗口（数秒）再停止引擎。随后重新以标准命令启动服务，等待 `/agentmemory/livez` 恢复 200，再重跑第 4.3 节的检索命令。探针记忆必须仍然返回。若使用了自定义 `--data-dir`，重启时必须传入相同目录。

4.6 无密钥模式的检索边界

无密钥模式下，捕获、索引与关键词检索完全可用，语义检索不可用。具体边界为：`memory_recall` 底层的 `mem::search` 使用 BM25；`smart_search` 在已有图谱数据时还能融合结构化图谱匹配。如需语义召回，在 `~/.agentmemory/.env` 中加入 `EMBEDDING_PROVIDER=local` 并重启，首次请求会下载 `Xenova/all-MiniLM-L6-v2` 模型（需要网络），此后推理全部在本地完成。该主题在第 6.4 节展开。

第二部分 进阶篇：理解并驾驭记忆系统

本部分深入 agentmemory 的内部机制：记忆有哪些类型、如何演化与遗忘、检索如何排序、日常有哪些操作入口、如何接入多个代理、以及如何按需配置。完成本部分后，读者能够解释任何一条记忆的来源与去向，并能针对自己的工作流调整系统行为。

第 5 章 记忆类型与生命周期

安装与接入完成之后，系统内部积累的是什么、如何演化、何时消亡，决定了对它的信任程度。本章解释六类记录的定义、产生方式与生命周期，并给出捕获与巩固的完整管线。完成本章后，读者能够说明任何一条记录从产生到消亡的全过程。

5.1 六类记录总览

记录类型	内容	产生方式
观测（observations）	钩子捕获的原始事件：提示词、工具调用、结果	钩子自动写入
记忆（memories）	持久化、带类型的结构性事实，版本链管理	`memory_save`、REST 或蒸馏自动生成
教训（lessons）	带置信度的短命令式规则	反思自动合成或显式保存
程序（procedures）	从重复模式提炼的工作流	巩固管线（需 LLM 提供方）
水晶（crystals）	单个会话的冻结摘要	会话结束时自动生成
图谱（graph）	实体与关系	会话结束时的结构化抽取

5.2 观测：一切的起点

钩子把每条提示词与每次工具调用作为观测发送到 `POST /agentmemory/observe`，附带事件标识（宿主代理自身的工具调用标识，无则用事件内容哈希）。服务端先把事件放入捕获收件箱（capture inbox），确认持久化后再移除；超时或重启后重放同一事件标识会被识别为重复而不再入库。服务不可达时，钩子把事件留在数据目录下的本地离线暂存区（offline spool），在下次启动或下次钩子调用时补交。观测在入库时被打上会话、项目、代理标识与来源通道（origin channel，取值为 user、agent、tool、import、shared 之一），来源在写入时固定，不可篡改。

5.3 记忆：版本链与相似提示

记忆是显式保存的持久事实，类型包括 pattern（模式）、preference（偏好）、architecture（架构）、bug（缺陷）、workflow（工作流）、fact（事实）。当新保存的内容与既有记忆高度相似时，发生取代（supersede）：新记录成为 v2 并链接到 v1，v1 退出全部检索索引但保留在版本链中可供追溯；相似度低于取代阈值时，保存结果返回 `similarTo` 提示，供调用方决定是否合并。检索永远只返回当前版本。

5.4 教训：置信度驱动的规则

教训是代理应当持续遵循的短规则。置信度在被强化时上升，每周未被使用时衰减；衰减到底且无强化的教训会被退休（retire）。教训召回使用专用的内存内 BM25 索引，排序依据为相关度乘以置信度乘以新近度。

5.5 程序：巩固管线的产物

当巩固（consolidation）开启且配置了 LLM 提供方时，会话结束触发巩固管线：跨至少两个会话出现的重复模式被总结为命名的程序，包含触发条件与步骤；再次检测到相同模式会强化学既有程序而非新建。

5.6 水晶与会话蒸馏

每个会话结束时生成一枚水晶：叙述、关键结果、涉及的文件与浮现的教训。水晶比原始观测活得久，保留策略（retention sweep）最终会修剪原始观测，而水晶保留下来。导入的 JSONL 转录也会自动生成水晶与教训。

5.7 图谱：无密钥也能建立

会话结束时的图谱抽取无条件运行：一个确定性结构化处理把文件与概念变为节点，把同一观测内的共现关系变为 `related_to` 边，因此图谱在零密钥下也能填充。设置 `GRAPH_EXTRACTION_ENABLED=true` 且有提供方密钥时，会在其上叠加一次 LLM 处理，增加带类型的关系。每个节点与边只保留最近 32 条来源观测的标识，防止长期运行的热点节点无限膨胀。

5.8 四层巩固模型

该模型模仿人脑记忆加工（包括睡眠巩固）：

层级	内容	类比
Working	工具调用产生的原始观测	短期记忆
Episodic	压缩后的会话摘要	发生了什么
Semantic	抽取的事实与模式	知道什么
Procedural	工作流与决策模式	怎么做

记忆随时间衰减（依据艾宾浩斯遗忘曲线），被频繁访问的记忆得到强化，陈旧记忆自动淘汰，检测到矛盾记录时会被解析处理。巩固衰减每个衰减周期最多执行一次，周期由 `CONSOLIDATION_DECAY_DAYS` 控制，默认 30 天。

5.9 捕获与压缩管线

PostToolUse 钩子触发后的完整管线：

```
PostToolUse 钩子触发
  -> SHA-256 去重（5 分钟窗口）
  -> 隐私过滤（剥离密钥、API key）
  -> 存储原始观测
  -> 默认合成压缩
     （仅在配置提供方且 AGENTMEMORY_AUTO_COMPRESS=true 时才用 LLM 压缩）
  -> 配置嵌入提供方后生成向量
  -> 写入 BM25 索引，向量可用时同时写入向量索引

Stop / SessionEnd 钩子触发
  -> 会话摘要
  -> 知识图谱抽取
  -> 槽位反思（SLOT_REFLECT_ENABLED=true 时）

SessionStart 钩子触发
  -> 载入项目画像（高频概念、文件、模式）
  -> 混合检索（BM25 + 向量 + 图谱）
  -> 按 token 预算（默认 2000）装填
  -> 注入对话
```

合成压缩（synthetic compression）指不调用语言模型的启发式处理，生成类型、标题、涉及文件与叙述；LLM 压缩则产出更丰富的事实与概念，但仅在配置密钥且 `AGENTMEMORY_AUTO_COMPRESS=true` 时启动。

第 6 章 检索机制详解

记忆积累之后，实用价值取决于能否被准确找回。本章解释检索的三路信号、融合排序方式、两个检索入口的差别与可调参数，并说明无密钥模式的检索边界。完成本章后，读者能够依据数据特征选择嵌入提供方并调整检索权重。

6.1 三路信号

每次查询融合三路信号：

信号	机制	生效条件
BM25	词干化关键词匹配，带同义词扩展	恒定开启
向量	稠密嵌入上的余弦相似度	配置嵌入提供方后
图谱	知识图谱按查询中的实体做图遍历	查询中检测到实体

RRF（Reciprocal Rank Fusion，倒数排名融合，k=60）将三路排名融合为一个总分；每个会话最多贡献 3 条结果，避免单会话霸屏；并列时按确定性规则解决。

6.2 两个检索入口的区别

`memory_recall`（底层为 `mem::search`）在向量索引已填充时使用 BM25 加向量的混合排序，否则纯 BM25。`smart_search` 在此之外还能融合图谱的结构化匹配，即使无密钥也可利用既有图谱数据。被取代的旧版本记忆不会出现在任何检索路径中。

6.3 嵌入提供方选择

提供方	模型	成本	说明
本地（推荐）	`all-MiniLM-L6-v2`	免费	首次下载模型后完全本地推理，比纯 BM25 提升约 8 个百分点的召回
Gemini	`gemini-embedding-001`	免费额度	支持 100 余种语言；已取代被弃用的 `text-embedding-004`（2026 年 1 月 14 日停服）
OpenAI	`text-embedding-3-small`	0.02 美元/百万 token	质量较高
Voyage AI	`voyage-code-3`	付费	针对代码优化
Cohere	`embed-english-v3.0`	免费试用	通用文本
OpenRouter	任意模型	视模型而定	多模型代理

启用本地嵌入：在 `~/.agentmemory/.env` 中加入 `EMBEDDING_PROVIDER=local` 并重启。远程提供方的密钥（如 `OPENAI_API_KEY`）会被自动检测，除非 `EMBEDDING_PROVIDER` 显式覆盖。官方配置文档给出的嵌入检测顺序为：`EMBEDDING_PROVIDER` 覆盖值、`GEMINI_API_KEY`、`OPENAI_API_KEY` 或 `OPENAI_EMBEDDING_API_KEY`、`VOYAGE_API_KEY`、`COHERE_API_KEY`、`OPENROUTER_API_KEY`，末位为本地模型；按 README 的说明，无密钥安装默认禁用向量，需显式设置 `EMBEDDING_PROVIDER=local` 启用本地语义召回。

6.4 中文等 CJK 语言的分词

BM25 原生支持希腊文、西里尔文、希伯来文、阿拉伯文与带变音符号的拉丁文。中文、日文、韩文记忆需安装可选分词器：

```
npm install @node-rs/jieba tiny-segmenter
```

未安装时退化为整段切分，并在标准错误输出一次性提示。

6.5 索引持久化与向量恢复

向量索引按桶保存，间隔至多为 `AGENTMEMORY_INDEX_SAVE_INTERVAL_MS`（默认 10 分钟）；间隔内每条向量的增删还会实时写入状态库中的小型待决日志（pending log），下次启动时无需调用嵌入提供方即可重放恢复。重放后仍缺向量的文档由后台补嵌（backfill）分批处理，批大小为 `AGENTMEMORY_VECTOR_BACKFILL_MAX`（默认 500），中断后下次启动续跑。`/agentmemory/status` 与查看器展示待决日志大小与补嵌状态。

6.6 检索权重调优

变量	作用	默认值
`BM25_WEIGHT`	BM25 路权重	0.4
`VECTOR_WEIGHT`	向量路权重	0.6
`AGENTMEMORY_GRAPH_WEIGHT`	smart-search 的图谱加成	0.2

第 7 章 日常操作：CLI、技能与会话导入

日常使用中，绝大多数操作经由命令行子命令与代理内技能完成，掌握它们的覆盖面即可摆脱对界面的依赖。本章按服务管理、捕获健康、历史导入、技能清单与引擎控制台五条线索整理操作入口，并给出一个可参考的日常流程。

7.1 服务管理命令

命令	作用
`agentmemory`	启动服务（工作进程、固定引擎、REST 3111、查看器 3113）
`agentmemory stop`	干净停止；先等待状态落盘窗口，`--force` 跳过 Docker 启发式保护
`agentmemory status`	显示连接状态、记忆数量、标志与健康
`agentmemory doctor`	交互式诊断与修复；`--all` 免提示全部修复，`--dry-run` 只预览
`agentmemory upgrade`	尽力升级本地依赖与 iii 运行时；只会拉取固定的 `iiidev/iii:0.22.1` 镜像，绝不安装更新引擎
`agentmemory remove`	干净卸载（pidfile、状态、.env、二进制）；`--force` 跳过确认，`--keep-data` 保留记忆数据
`agentmemory init`	把内置 `.env.example` 复制为 `~/.agentmemory/.env`（已存在则跳过）

7.2 demo 与捕获健康

`demo [--serve]` 见第 4.1 节。`capture` 子命令查看捕获健康度：

```
agentmemory capture           # 查看离线暂存与服务收件箱（待处理、重试中、死信）
agentmemory capture --drain   # 立即发送暂存的观测
agentmemory capture --json    # 机器可读输出
```

死信（dead letter）指重试 `AGENTMEMORY_CAPTURE_MAX_ATTEMPTS`（默认 5 次）后仍失败的捕获，会显示在 `/agentmemory/status` 中。

7.3 会话导入：import-jsonl

把既有的 Claude Code 会话转录导入记忆库：

```
agentmemory import-jsonl                          # 导入 ~/.claude/projects 下全部文件
agentmemory import-jsonl ~/.claude/projects/-my-project/abc123.jsonl   # 导入单个文件
```

每份转录经 `mem::replay::import-jsonl` 等函数入库，建立检索索引，来源标记为 `import` 通道，并自动提炼水晶与教训。`--max-files <N>` 覆盖扫描上限（默认 200，最大 1000）。

重要提醒：Claude Code 的 `cleanupPeriodDays` 设置（位于 `~/.claude/settings.json`，默认 30）会自动删除超过该天数的 JSONL 转录。若在陈旧历史上首次导入，超过 30 天的记录在导入前已被删除。对策有三种：定时执行 `import-jsonl`、调高 `cleanupPeriodDays`、或改用钩子自动捕获（默认插件路径），使记录在会话进行时就落库。

7.4 十七个技能

技能分为两类。九个可调用技能：

技能	用途
`/remember`	保存到长期记忆
`/recall`	检索记忆
`/recap`	概括当前会话进展
`/handoff`	在代理间交接工作
`/forget`	删除观测或会话
`/lesson`	保存教训
`/commit-context`	提供提交相关上下文
`/commit-history`	查看提交历史记忆
`/session-history`	最近的会话摘要

八个参考技能（代理按需自动加载）：`memory-discipline`（记忆使用纪律）、`agentmemory-mcp-tools`、`agentmemory-rest-api`、`agentmemory-config`、`agentmemory-agents`、`agentmemory-hooks`、`agentmemory-architecture`、`write-agentmemory-skill`（编写自定义技能的指南）。参考技能内嵌由源码生成的数据表，避免文档漂移。

7.5 iii 控制台

查看器展示代理记住了什么，iii 控制台展示代理做了什么：

```
agentmemory console
```

该命令用固定引擎的 `iii console` 按当前实例解析出的端口启动控制台，默认服务于查看器端口加一，即 `http://localhost:3114`；`--console-port N` 可改端口，`--enable-flow` 开启实验性的架构图页面。控制台提供 Workers（工作进程与指标）、Functions（直接以 JSON 载荷调用任意函数）、Triggers（重放 HTTP、cron、事件触发器）、States（KV 浏览器，可编辑会话、槽位、生命周期计时器与嵌入索引）、Streams（实时事件监视）、Queues（持久队列与死信管理）、Traces（OpenTelemetry 瀑布图，可按 trace 过滤）、Logs（结构化日志）、Config（运行时配置）等页面。

两点注意：引擎自带的可观测性 worker 已默认启用（`exporter: memory`，采样率 0.1），如需导出到 Jaeger、Honeycomb 或 Grafana Tempo，将 `exporter: memory` 改为 `exporter: otlp` 并按 iii 文档设置收集端点；控制台本身无认证，必须保持绑定 127.0.0.1，切勿暴露到公网。

7.6 一个典型的日常流程

以下是接入完成后一天的典型使用序列，可作为习惯模板：

```
# 1. 早晨启动服务（若未设开机自启）
agentmemory

# 2. 在代理中正常工作；钩子在后台自动捕获，无需干预

# 3. 需要交接或复盘时，在代理内使用技能
#    /recap 生成阶段摘要，/handoff 生成交接材料

# 4. 需要审计或排障时
agentmemory capture
agentmemory doctor
```

第 8 章 多 agent 接入：MCP 标准配置块与跨 agent 共享

单一代理的记忆价值有限，复利来自多个代理共享同一份记忆。本章说明接入面的三层原理、21 个适配器的差异、主流代理的具体要点，以及沙箱环境与 Windows 场景的处理方式。

8.1 一套记忆服务多个代理的原理

所有代理连向同一个 3111 端口的服务，记忆集中存储，因此 Claude Code 中形成的架构决策，Cursor 会话开始时同样可用。接入面有三层：钩子（自动捕获，覆盖 Claude Code、OpenCode、Codex、Droid 等）、MCP 客户端（按需调用工具，覆盖 Cursor、Gemini CLI、Zed 等）、REST（覆盖 Aider 等直接调用 HTTP 的程序）。

8.2 21 个适配器概览

代理	接入形态	connect 写入位置
Claude Code	MCP + 钩子	`~/.claude.json`；`--with-hooks` 合并到 `~/.claude/settings.json`
Codex CLI	MCP + 钩子	`~/.codex/config.toml`；`--with-hooks` 写全局 `~/.codex/hooks.json`
Cursor	插件（7 钩子 + 17 技能 + MCP）或仅 MCP	`.cursor-plugin/` 或 `~/.cursor/mcp.json`
GitHub Copilot CLI	MCP	`~/.copilot/mcp-config.json`
Devin	MCP + 6 钩子或插件	`~/.config/devin/config.json` 或 `plugin/.devin-plugin/`
Gemini CLI	MCP	`~/.gemini/settings.json`
OpenCode	MCP + 捕获插件	`opencode.json` 顶层 `mcp` 键；插件在 `plugin/opencode/`
OpenClaw	MCP（原生记忆槽插件另装）	`~/.openclaw/openclaw.json`
Hermes Agent	MCP（手工）	`~/.hermes/config.yaml` 的 `mcp_servers` 块
pi	原生扩展	`~/.pi/agent/extensions/agentmemory`（下次启动自动发现）
Warp	MCP	`~/.warp/.mcp.json`
Zed	MCP	`settings.json` 的 `context_servers`
Qwen Code	MCP	`~/.qwen/settings.json`
Antigravity / Antigravity CLI	MCP（+ 钩子）	`~/.gemini/config/mcp_config.json`
Kiro	MCP	`~/.kiro/settings/mcp.json`
Cline	MCP	`~/.cline/mcp.json`
Continue	MCP	`~/.continue/config.yaml`（无 yaml 时用 config.json）
Droid	MCP + 钩子	`~/.factory/mcp.json`
DeepSeek Harness	MCP + 钩子	`$DSH_HOME/cordis.patch.yml`
OpenHuman	原生（手工）	接 REST 3111
Aider	REST API	直接调用 `http://localhost:3111`

钩子型代理自动捕获观测；仅 MCP 的代理会被写入一条记忆使用指引（如全局 GEMINI.md、Cursor 的 `.cursor/rules/*.mdc`），促使代理主动调用工具。

8.3 主流代理接入要点

Codex CLI 插件路径：先启动服务，再注册市场并安装插件：

```
codex plugin marketplace add rohitg00/agentmemory
codex plugin add agentmemory@agentmemory
```

插件注册 MCP、6 个生命周期钩子（SessionStart、UserPromptSubmit、PreToolUse、PostToolUse、PreCompact、Stop）与 17 个技能。钩子分发能力经 Codex CLI 0.150.1 验证；Subagent、SessionEnd、Notification 等事件仅 Claude Code 支持。若 MCP 已接入，`connect codex --with-hooks` 需追加 `--force` 才会安装钩子。为避免重复捕获，原生插件钩子与全局钩子副本二者只启用其一。

OpenCode：MCP 配置见第 3.3 节；捕获插件复制两处文件即可：

```
mkdir -p ~/.config/opencode/plugins
cp plugin/opencode/agentmemory-capture.ts ~/.config/opencode/plugins/
cp plugin/opencode/commands/*.md ~/.config/opencode/commands/
```

插件含 22 个捕获钩子（覆盖会话生命周期、消息、工具与错误）与 `/recall`、`/remember` 两个斜杠命令；项目归属按会话判定，单个 OpenCode 进程跨多个仓库时各会话分别归属各自项目。

OpenClaw 与 Hermes：

```
agentmemory connect openclaw    # 安装原生插件与技能，启用记忆槽
agentmemory connect openclaw --with-mcp   # 可选：追加更宽的 MCP 工具面
agentmemory connect hermes      # 安装提供方与技能
hermes memory setup             # 在 Hermes 侧配置凭据
hermes memory status
```

两者的 MCP 工具访问后端时不带原生插件的档案过滤，仅建议在可信且有意共享的后端上启用 `--with-mcp`。

8.4 通用 MCP 块与配置文件对照

第 3.3 节的标准块适用于全部 `mcpServers` 形态的宿主；对照表见第 8.2 节，各宿主的细粒度差异（如 Continue 的数组形态、Goose 的 `extensions:` 加 `cmd` 键）以官方各集成页为准。

8.5 沙箱客户端与远程部署

Flatpak、Snap 等沙箱化客户端无法访问宿主的 localhost 时，在 MCP 配置的 env 块中追加 `"AGENTMEMORY_FORCE_PROXY": "1"`，并把 `AGENTMEMORY_URL` 指向沙箱实际可达的地址（例如局域网 IP）。远程部署的完整讨论见第 12 章。

8.6 Windows 接入差异

原生 Windows 上自动化 connect 仅支持 `copilot-cli`；Claude Code、Codex、Cursor 等需按第 3.3 节手工写入各代理的 Windows 配置。WSL 内执行 connect 仅当目标代理也安装在同一个 WSL 环境时才有意义，不会改写 Windows 宿主侧代理的配置。

第 9 章 配置调优：环境变量、LLM 与嵌入提供方、上下文注入预算

默认配置足以运行，调优才能贴合具体工作流。本章按提供方、注入预算、行为开关、捕获范围、存储耐久与桥接六个方面梳理最常用的环境变量；官方配置文档共列出 99 个变量，未覆盖部分以官方 Configuration 页为准。

9.1 配置文件与总体原则

全部配置经环境变量读取，来源文件为 `~/.agentmemory/.env`； `agentmemory init` 可从内置 `.env.example` 生成。默认无密钥即可运行；REST API 的持有者令牌认证默认开启，与 LLM 密钥是两回事。官方配置文档列出 99 个环境变量，本文只覆盖调优中最常用的一批，全量以官方 Configuration 页为准。修改 `.env` 后需重启服务生效； `agentmemory doctor` 可验证守护进程实际读取了哪些标志。

9.2 LLM 提供方

检测顺序： `OPENAI_API_KEY`、 `MINIMAX_API_KEY`、 `ANTHROPIC_API_KEY`、 `GEMINI_API_KEY`、 `OPENROUTER_API_KEY`，全部未设时为 noop（不启用 LLM 功能）。配置密钥后解锁的功能包括 LLM 摘要、反思、巩固、图谱带类型关系抽取与结晶化。

变量	作用	默认
`OPENAI_API_KEY`	激活 OpenAI 兼容提供方（DeepSeek、LiteLLM、vLLM、LM Studio、经 `/v1` 的 Ollama）与 OpenAI 嵌入	无
`OPENAI_BASE_URL`	OpenAI 兼容提供方的地址覆盖	`https://api.openai.com`
`OPENAI_MODEL`	OpenAI 兼容聊天模型	`gpt-5.6-luna`
`ANTHROPIC_API_KEY`	Anthropic 提供方	无
`ANTHROPIC_MODEL`	Anthropic 模型	`claude-sonnet-5`
`GEMINI_API_KEY`	Gemini 提供方（优先于 `GOOGLE_API_KEY`）	无
`OPENROUTER_API_KEY`	OpenRouter 提供方	无
`MINIMAX_API_KEY`	MiniMax 提供方	无
`MAX_TOKENS`	压缩与摘要调用的补全 token 上限	4096
`AGENTMEMORY_LLM_TIMEOUT_MS`	出站 LLM 与嵌入调用的共享超时	60000
`FALLBACK_PROVIDERS`	主提供方报错后依次尝试的回退链	无
`AGENTMEMORY_ALLOW_AGENT_SDK=true`	经 agent-sdk 子会话启用 Claude 订阅回退，默认关闭	关闭

`OPENAI_API_KEY_FOR_LLM=false` 可让密钥只用于嵌入而跳过 LLM 自动检测。

9.3 上下文注入

`AGENTMEMORY_INJECT_CONTEXT=true` 开启会话开始的记忆注入（默认关闭）。注入块按固定顺序组装：固定的记忆槽位（pinned slots）优先，其次是项目画像（高频概念、关键文件、约定、常见错误），然后是最多 10 条教训（按置信度排序，项目级教训优先于全局），最后是同项目最近会话的摘要。所有块按 `TOKEN_BUDGET`（默认 2000）装填，装不下的整块丢弃而非截半。`AGENTMEMORY_PRE_COMPACT_BUDGET`（默认 1500）控制压缩前的注入预算，设 0 禁用。

9.4 行为开关

变量	作用	默认
`AGENTMEMORY_AUTO_COMPRESS=true`	用 LLM 压缩每批观测，需提供方密钥	关闭（用合成压缩）
`CONSOLIDATION_ENABLED=true`	运行四层巩固管线	关闭
`CONSOLIDATION_DECAY_DAYS`	未强化记忆开始衰减的天数	30
`GRAPH_EXTRACTION_ENABLED=true`	开启图谱 LLM 带类型关系抽取；无密钥结构化抽取始终运行	关闭
`AGENTMEMORY_REFLECT=true`	周期性从记忆自动合成教训	关闭
`AGENTMEMORY_DROP_STALE_INDEX=true`	维度守卫触发时在启动时丢弃磁盘上的 BM25 与向量索引	关闭

9.5 捕获范围控制

变量	作用	默认
`AGENTMEMORY_CAPTURE_ALLOW`	白名单：仅捕获列出的工具名或通配模式	未设（全捕获）
`AGENTMEMORY_CAPTURE_DENY`	追加黑名单，叠加在内置排除项（`memory_*`、`toolsearch`、`listmcpresources`、`fetchmcpresource`）之上	未设
`AGENTMEMORY_CAPTURE_OUTPUT_MAX`	每条观测保留的工具输出字符上限	8000
`AGENTMEMORY_CAPTURE_SPOOL`	离线暂存开关	开启
`AGENTMEMORY_CAPTURE_SPOOL_MAX_BYTES`	单主机暂存文件上限	5 MiB

9.6 存储与耐久

变量	作用	默认
`AGENTMEMORY_STATE_BACKEND`	`file` 或 `redis`；切换后端需导出再导入，不做在线迁移	`file`
`AGENTMEMORY_STATE_SAVE_INTERVAL_MS`	文件态落盘间隔，硬杀最多丢失该窗口内的写入	2000
`AGENTMEMORY_INDEX_SAVE_INTERVAL_MS`	检索索引保存的最小间隔；关机与删除时立即保存	600000
`SNAPSHOT_ENABLED=true`	周期性快照状态与事件流存储	关闭
`SNAPSHOT_DIR`	快照目录	`~/.agentmemory/snapshots`
`SNAPSHOT_INTERVAL`	快照间隔（秒）	3600

9.7 Claude 桥接与 Obsidian 导出

`CLAUDE_MEMORY_BRIDGE=true` 与 `CLAUDE_PROJECT_PATH`（必填，指定项目目录）开启向 Claude Code 记忆文件的镜像：`memory_claude_bridge_sync` 仅导出该项目范围的最新记忆（不含无范围记忆），读取方向则继续读取既有 MEMORY.md。导出只在文件不存在时创建 MEMORY.md；内容一致时返回 `changed: false`，不一致时报错；需重新生成时先把现有文件移走。
`CLAUDE_MEMORY_LINE_BUDGET`（默认 200）限制桥接文件行数。`OBSIDIAN_AUTO_EXPORT=true` 在每次巩固后自动导出记忆到 Obsidian 库。

第三部分 高级篇：集成、部署与精通

本部分面向把 agentmemory 用于个人生产或团队协作的读者：程序化集成、团队共享与治理、生产部署、日常运维，以及以基准复现和最佳实践收束的精通之路。

第 10 章 程序化集成：REST API、MCP 工具与 SDK

代理之外的工具、脚本与其他服务同样需要读写记忆，这需要程序化接口。本章覆盖 REST API 家族、认证与请求规则、MCP 工具体系与 SDK 直接调用四个集成层。完成本章后，读者能够把记忆能力接入自建系统。

10.1 REST API 家族总览

全部路由位于 3111 端口的 `/agentmemory/*` 前缀下，完整接口共 138 个，权威来源为仓库内 `src/triggers/api.ts`。按家族划分：

家族	代表路由	用途
健康	`livez`、`health`、`status`	存活、子系统健康、综合状态
捕获	`observe`、`capture/retry`、`capture/drain`	摄入钩子事件、重试与清空
会话	`session/start`、`session/end`、`sessions`、`summarize`、`commits`	会话生命周期与 git 提交关联
观测	`observations`、`observations/locate`、`context`	列出与定位观测、注入上下文
记忆	`remember`、`memories`、`memories/:id`、`semantic`、`procedural`、`evolve`、`evict`	各层记忆的存取与演化
检索	`search`、`smart-search`、`timeline`、`profile`、`patterns`	关键词、混合、时序与模式召回
教训	`lessons`、`lessons/search`、`lessons/strengthen`	保存、召回、强化与退休
槽位	`slots`、`slot/append`、`slot/replace`	可编辑、限量型的记忆单元
Actions	`actions`、`leases/*`、`signals/*`、`routines/*`	工作项、依赖与协调层
结晶与重放	`crystals`、`replay/load`、`replay/import-jsonl`	会话压缩与转录重放
图谱	`graph/query`、`graph/stats`、`graph/compact`	图谱查询与溯源修剪
团队与 mesh	`team/share`、`team/feed`、`mesh/*`	见第 11 章
治理	`forget`、`auto-forget`、`governance/*`、`audit`	见第 11 章
导出导入	`export`、`import`、`obsidian/export`、`snapshot/*`	全量 JSON、Obsidian 库、快照
其他	`diagnostics`、`diagnostics/heal`、`branch/*`	自诊断、自修复、分支感知

10.2 认证与请求规则

持有者令牌认证默认开启，密钥解析顺序为环境变量、`~/.agentmemory/.env`、生成的 `~/.agentmemory/secret`；显式设置的 `AGENTMEMORY_SECRET` 恒优先。三个请求规则：携带请求体的写操作（POST、PUT、PATCH、DELETE）必须声明 `Content-Type: application/json`；若请求带 `Origin` 头，其值必须是配置端口的回环来源、查看器自身来源或 `VIEWER_ALLOWED_ORIGINS` 列表成员（CLI、钩子、MCP、curl 不发 Origin，不受影响）；`/compress-file`、`/replay/import-jsonl`、`/graph/import-graphify`、Obsidian 导出与 `/migrate` 只接受数据目录、`~/.agentmemory` 或 `AGENTMEMORY_IMPORT_ROOT` 内的路径（多目录以冒号分隔，Windows 用分号）。

10.3 常用端点实战

综合状态报告：

```
curl -H "Authorization: Bearer $(cat ~/.agentmemory/secret)" \
  http://localhost:3111/agentmemory/status
```

带项目与代理范围的保存与检索（`project` 与 `agentId` 用于范围限定）：

```
curl -X POST http://localhost:3111/agentmemory/remember \
  -H "Authorization: Bearer $(cat ~/.agentmemory/secret)" \
  -H 'Content-Type: application/json' \
  -d '{"content":"数据库连接池固定为 20","project":"acme-api","agentId":"claude-code"}'
```

列出记忆（默认最新在前，支持 `nextCursor` 分页与 `facets`）：

```
curl -H "Authorization: Bearer $(cat ~/.agentmemory/secret)" \
  "http://localhost:3111/agentmemory/memories?limit=20"
```

删除记忆（返回值只统计真实删除；不存在的标识列入 `notFound`，失败的列入 `failed`）：

```
curl -X POST http://localhost:3111/agentmemory/forget \
  -H "Authorization: Bearer $(cat ~/.agentmemory/secret)" \
  -H 'Content-Type: application/json' \
  -d '{"ids":["<记忆标识>"]}'
```

示例请求体中的字段名未在本文核实范围内逐字验证，以官方 REST API 参考页为准。

10.4 MCP 工具体系

工具面分三档，由服务端标志 `AGENTMEMORY_TOOLS` 控制：`core` 仅暴露 8 个核心工具（`memory_save`、`memory_recall`、`memory_consolidate`、`memory_smart_search`、`memory_sessions`、`memory_diagnose`、`memory_lesson_save`、`memory_reflect`）；基础集为注册表中 14 个基础工具；默认 `all` 暴露全部 54 个。除工具外还有 6 个资源（如 `agentmemory://status`、`agentmemory://project/{name}/profile`）、3 个提示（`recall_context`、`session_handoff`、`detect_patterns`）。

关于 `@agentmemory/mcp` shim 包的一个重要行为：它是瘦代理，仅在能通过 `AGENTMEMORY_URL` 连上运行中的 agentmemory 服务时暴露全部 54 个工具（代理模式）；连不上时回退为 7 个工具的本地集（`memory_save`、`memory_recall`、`memory_smart_search`、`memory_sessions`、`memory_export`、`memory_audit`、`memory_governance_delete`）。`AGENTMEMORY_TOOLS` 是服务端标志，写在 shim 的 env 块中无效。若在 Cursor、OpenCode 或 Gemini CLI 中只看到 7 个工具，应启动 `npx -y @agentmemory/agentmemory@latest` 并设置 `AGENTMEMORY_URL=http://localhost:3111`。

10.5 SDK 编程访问

agentmemory 把核心操作注册为 iii 函数（`mem::remember`、`mem::observe`、`mem::context`、`mem::smart-search`、`mem::forget`），任何具备 iii SDK 的语言可经 `ws://localhost:49134` 直接调用：

```
pip install iii-sdk      # Python
cargo add iii-sdk        # Rust
npm  install iii-sdk     # Node
```

Python 示例：

```
from iii import register_worker

iii = register_worker("ws://localhost:49134")
iii.connect()

iii.trigger({
    "function_id": "mem::smart-search",
    "payload": {"project": "demo", "query": "how do tokens refresh"},
})
```

完整示例见仓库 `examples/python/`。无 iii 运行时的宿主可继续使用 3111 端口的 REST。

第 11 章 团队协作与数据治理

记忆长期积累之后，治理机制决定其可信度与寿命。本章覆盖单实例内的团队共享、跨实例的 mesh 同步、删除遗忘与审计、导出导入备份，以及贯穿始终的隐私与溯源机制。

11.1 团队共享：单实例内的团队动态

设置 `TEAM_MODE=shared`、`TEAM_ID` 与 `USER_ID` 后，记忆按（团队、用户）二元组确定范围。

接口	作用
`memory_team_share`（MCP）或 `POST /agentmemory/team/share`	把记忆、模式或观测（观测需附 `sessionId`）分享进团队动态，附分享人与时间
`memory_team_feed`（MCP）或 `GET /agentmemory/team/feed`	全体成员的最近分享，最新在前
`GET /agentmemory/team/profile`	动态中出现的成员与分享计数

共享文本经过与其他写入相同的机密清洗器，API 密钥与凭据在进入动态前被删除。经共享收到的记录带 `shared` 来源通道，检索时可与本人写入区分。

11.2 实例间 mesh 同步

mesh 用于在多个独立 agentmemory 实例之间同步数据。先以名称与 URL 注册对端：

```
curl -X POST http://localhost:3111/agentmemory/mesh/peers \
  -H "Authorization: Bearer $AGENTMEMORY_SECRET" \
  -H 'Content-Type: application/json' \
  -d '{"name":"laptop-2","url":"https://mem.example.com"}'
```

请求体字段名以官方 Sharing 文档为准。对端 URL 必须是公网 http 或 https 地址，localhost、私有地址段与链路本地地址（包括经 DNS 解析后）都会被拒绝。`POST /agentmemory/mesh/sync` 以 `direction: "push" | "pull" | "both"` 对单个或全部对端执行推送、拉取或双向同步；默认同步范围为 `memories`、`actions`、`semantic`、`procedural`、`relations`、`graph:nodes`、`graph:edges`，可收窄并按项目过滤。冲突按记录时间戳逐条最后写入优先，在键级锁下解决。每个 mesh 路由都要求两端显式设置 `AGENTMEMORY_SECRET`（自动生成的文件不算数，缺失时返回 503）。全部注册、同步与接收操作均写入审计。

11.3 治理：删除、遗忘与审计

端点或工具	用途
`POST /agentmemory/forget`	按标识删除，返回 `notFound` 与 `failed` 明细
`POST /agentmemory/auto-forget`	触发自动遗忘（TTL 过期、矛盾检测、重要性淘汰）
`POST /agentmemory/governance/bulk-delete`	按过滤条件批量删除；拒绝空过滤器与不支持的键
`GET /agentmemory/audit`	审计日志，返回 `{ entries, legacyFrozen, legacyFrozenBytes }`
`GET /agentmemory/verify`	溯源验证，核对记忆与来源观测的一致性

删除属于不可逆操作：执行前应确认过滤条件，重要数据先走第 11.4 节的导出。

11.4 导出、导入与 Obsidian

全量导出与导入：

```
curl -H "Authorization: Bearer $(cat ~/.agentmemory/secret)" \
  http://localhost:3111/agentmemory/export > backup.json
```

导出体积超限时返回 413 状态码与 `{ oversized, bytes, limitBytes, stoppedAt }`。导入使用 merge 策略：

```
jq -n --slurpfile d backup.json '{exportData: $d[0], strategy: "merge"}' | \
  curl -fsS -H "Authorization: Bearer $(cat ~/.agentmemory/secret)" \
    -X POST http://localhost:3111/agentmemory/import \
    -H 'Content-Type: application/json' -d @-
```

`jq` 为命令行 JSON 处理工具，需单独安装。另有 `POST /agentmemory/obsidian/export` 导出 Obsidian 库格式，以及 `snapshot/create` 与 `snapshot/restore` 两个快照端点（记忆状态的版本化、回滚与对比）。

11.5 版本演化

`POST /agentmemory/evolve` 对既有记忆执行演化操作；对冲突、非法与缺失输入分别返回 409、400 与 404，不再一律成功。结合第 5.3 节的版本链，记忆的每次修订都可追溯。

11.6 隐私与溯源

三个机制共同构成隐私与溯源保障：写时来源通道（user、agent、tool、import、shared）在捕获、保存与导入时盖章且不可变；机密清洗在存储前剥离 API 密钥、令牌与 `<private>` 标签内容；引用溯源允许把任何记忆回溯到来源观测。`agentId` 贯穿保存与检索（REST、MCP 与检索索引），支持共享或隔离两种模式。

第 12 章 生产部署：Docker、云模板、Redis 后端与安全加固

从个人机器走向团队可用，需要解决部署形态、共享后端与安全暴露三个问题。本章给出四种部署形态的选择依据与具体步骤，并以一份安全清单收束。完成本章后，读者能够把 agentmemory 部署为团队可用的常驻服务。

12.1 部署形态选择

形态	适用	关键点
本地单机	个人使用	默认形态，第 2 章已完成
Docker Compose	单服务器自托管	用仓库自带 `docker-compose.yml`
云模板	无本地常驻机器	fly.io、Railway、Render、Coolify 四种模板
Redis 后端	多进程并发写同一实例	`AGENTMEMORY_STATE_BACKEND=redis`

12.2 Docker Compose

仓库的 `docker-compose.yml` 拉取 `iiidev/iii:0.22.1` 镜像，把 `iii-config.docker.yaml` 以只读方式挂载到 `/app/config.yaml`。该配置把 HTTP、事件流与引擎端口绑定 `0.0.0.0` 并把状态存放在 `/data`，需挂载可写卷。启动入口脚本会用部署调优配置覆盖 npm 内置的回环配置、生成 HMAC 密钥、经 `gosu` 降权到 `node` 用户后再执行 CLI。

另一条路径是 `AGENTMEMORY_USE_DOCKER=1` 让 CLI 走 Docker。注意捆绑配置在容器内也绑定 127.0.0.1，发布端口将无法访问；此时应把 `AGENTMEMORY_III_CONFIG` 指向绑定 `0.0.0.0` 的配置（内置 `iii-config.docker.yaml` 即是），并保持 `AGENTMEMORY_SECRET` 已设置。

12.3 云模板

四种一键模板各自提供自足的 Dockerfile（从 npm 拉取 `@agentmemory/agentmemory` 并从官方 `iiidev/iii` 镜像复制引擎二进制）：

模板	特点
fly.io	单机，`auto_stop_machines = "stop"`，空闲成本最低
Railway	Hobby 套餐固定月费，卷在控制台配置
Render	Blueprint 流程，付费方案自动磁盘快照
Coolify	自有 VPS 上的自托管，同一套 Docker Compose 栈

只发布 3111 端口；查看器 3113 在容器内保持回环绑定，各模板 README 记载了经 SSH 隧道访问查看器的方法。

12.4 Redis 状态后端

文件后端适合单用户本地安装；共享守护进程与多个并发写者应换用 Redis，它把每个键存为 Redis 哈希字段，代价是每次操作多一次网络往返：

```
# ~/.agentmemory/.env
AGENTMEMORY_STATE_BACKEND=redis
AGENTMEMORY_REDIS_URL=redis://localhost:6379
```

约束与建议如下，均为官方文档明确列出的行为：

- 固定引擎 0.22.1 的 Redis 客户端不支持 TLS，只能用明文 `redis://`；`rediss://`（多数托管 Redis 默认）会连接失败。明文传输意味着 Redis 密码与全部记忆在网络上可见，应指向本机或可信私网内的 Redis，跨网段用 stunnel、SSH 隧道或 VPN 加密；
- 每个 `--instance` 必须对应独立的 Redis 服务器：引擎的键前缀固定，多实例指向同一库会互相覆盖；换库编号也无效，因为查看器的实时事件走单一 pub/sub 频道，不区分库编号；
- Redis 重启后引擎停止向查看器转发实时事件，直到 agentmemory 重启；数据读写不受影响。健康监控每 30 秒发一次测试事件，失败时状态页会给出修复建议（重启 agentmemory）；
- 推荐配置：`appendonly yes`（默认快照策略在崩溃时可能丢分钟级写入），`maxmemory-policy noeviction`（`allkeys-lru` 等策略会在内存满时静默丢弃记忆）；
- 引擎会把展开后的 Redis URL 持久化到 `~/.agentmemory/config/` 下的文件，共享主机上应 `chmod 700 ~/.agentmemory`，并优先使用权限最小的 Redis ACL 账户；
- 后端切换不是自动迁移：先在旧后端导出，再在新后端导入（脚本见第 13 章清单）。

12.5 远程访问与安全加固

生产部署的安全清单：

- 远程（非回环）访问任何端点都必须显式设置 `AGENTMEMORY_SECRET`，自动生成的密钥文件只发给回环地址；
- mesh 同步两端都要显式密钥；
- 仅发布确实需要的端口；查看器与 iii 控制台保持回环绑定，需要时经 SSH 隧道访问；
- `VIEWER_ALLOWED_ORIGINS` 可追加允许经 REST 与查看器写入的来源；

- 引擎匿名遥测默认关闭（除非显式设置 `III_TELEMETRY_ENABLED=true`），引擎启动时带 `--no-update-check`，不会在启动时查询 GitHub。

12.6 快照与备份

`SNAPSHOT_ENABLED=true` 开启周期快照（默认每小时，目录 `~/.agentmemory/snapshots`）；`POST /agentmemory/snapshot/create` 与 `snapshot/restore` 支持手工创建与恢复。此外文件后端本身有秒级落盘窗口（`AGENTMEMORY_STATE_SAVE_INTERVAL_MS`），硬杀进程最多丢失该窗口内的写入。

第 13 章 运维与故障排除

长期运行的系统必然经历故障、升级与数据搬迁。本章给出捕获健康诊断方法、升级流程与配套事项、一份可直接执行的后端迁移脚本，以及按症状索引的排查表。

13.1 捕获健康诊断

捕获链路有三个可观测点：服务收件箱（待处理、重试中、死信）、钩子侧离线暂存、以及 `/agentmemory/status` 的汇总报告。排查顺序建议为：

```
agentmemory capture            # 查看两端队列状态
agentmemory capture --drain    # 手动清空暂存
agentmemory doctor             # 交互式诊断与修复
```

死信说明某事件重试 5 次仍失败，可经 `capture/retry` 重试；收件箱满（默认上限 10000 条）时新观测会被以可重试的 503 拒绝，钩子会转入暂存。

13.2 升级流程

升级 agentmemory 本体：

```
npx -y @agentmemory/agentmemory@latest upgrade
```

`upgrade` 会更新 JavaScript 依赖并拉取固定的 `iiidev/iii:0.22.1` 镜像，绝不安装未固定或更新的引擎；引擎与 `iii-sdk`、`@iii-dev/helpers` 必须同版本，启动时会拒绝采纳端口上运行的其他版本引擎。除非确认目标版本在线协议上兼容，否则不要用 `AGENTMEMORY_III_VERSION` 覆盖。

升级后两件配套事项：重跑 `connect <agent> --with-hooks` 刷新钩子的绝对路径（第 3.1 节）；若走的是 Claude Code 的 MCP 独立配置路径，同样重跑 `agentmemory connect claude-code --with-hooks`。

13.3 备份与后端迁移脚本

从文件后端迁移到 Redis（或反向）的完整序列，兼容 bash 与 zsh：

```
# 0. 未导出密钥时先读取生成的密钥
AGENTMEMORY_SECRET="${AGENTMEMORY_SECRET:-$(cat ~/.agentmemory/secret 2>/dev/null)}"

# 1. 在旧后端上、服务仍在运行时导出
if [ -n "${AGENTMEMORY_SECRET:-}" ]; then
  curl -fsS -H "Authorization: Bearer $AGENTMEMORY_SECRET" \
    http://localhost:3111/agentmemory/export > backup.json
else
  curl -fsS http://localhost:3111/agentmemory/export > backup.json
fi

# 2. 校验导出文件可用后再切换后端
jq -e '.version and .exportedAt' backup.json > /dev/null || {
  echo "backup.json 不是有效的导出文件，终止切换" >&2
  exit 1
}

# 3. 修改 AGENTMEMORY_STATE_BACKEND（必要时含 AGENTMEMORY_REDIS_URL）并重启，
#    然后导入
if [ -n "${AGENTMEMORY_SECRET:-}" ]; then
  jq -n --slurpfile d backup.json '{exportData: $d[0], strategy: "merge"}' | \
    curl -fsS -H "Authorization: Bearer $AGENTMEMORY_SECRET" \
      -X POST http://localhost:3111/agentmemory/import \
      -H 'Content-Type: application/json' -d @-
fi
```

注意一个 zsh 与 bash 的差异陷阱：形如 `AUTH=(${AGENTMEMORY_SECRET:+-H "Authorization: Bearer $AGENTMEMORY_SECRET"})` 的数组展开在 zsh 中会把整个头部当作一个畸形词，导致请求一律 401；上面脚本采用逐分支写法以规避该问题。

13.4 常见问题排查表

症状	原因	处理
手工 curl 请求 401	未携带持有者令牌	加 `-H "Authorization: Bearer $(cat ~/.agentmemory/secret)"`
MCP 客户端只见 7 个工具	shim 未连上服务	启动完整服务并确认 `AGENTMEMORY_URL`
`npx` 跑的是旧版本	npx 按版本缓存	用 `npx -y @agentmemory/agentmemory@latest` 或全局安装
全局安装报 EACCES	npm 全局目录权限	修复 npm 前缀权限或改用 npx 方式
端口冲突	四端口被占用	`netstat`（Windows）或 `lsof`（macOS/Linux）定位后释放，或 `--port <N>`、`--instance <N>`
无法启动 iii-engine（Windows）	未装 `iii.exe` 且未用 Docker	第 2.3 节方式一或方式二
引擎启动但 REST 无响应	端口占用或引擎异常退出	加 `--verbose` 查看引擎错误输出
中文检索效果差	缺少 CJK 分词器	`npm install @node-rs/jieba tiny-segmenter`
语义查询返回零结果	无密钥模式无向量	设置 `EMBEDDING_PROVIDER=local` 并重启，等待首次模型下载完成
查看器数字停止更新（Redis 后端）	Redis 重启后事件转发中断	重启 agentmemory（数据无损）
升级后钩子失效	钩子路径内嵌旧版本号	重跑 `connect <agent> --with-hooks`
会话历史导入不全	Claude Code `cleanupPeriodDays` 已清理旧转录	第 7.3 节的三种对策

13.5 卸载

```
agentmemory remove              # 交互确认后卸载
agentmemory remove --force      # 跳过确认
agentmemory remove --keep-data  # 卸载但保留记忆数据
```

第 14 章 精通之路：基准复现、生态组合与最佳实践

精通的标志不是记住更多命令，而是能够独立验证官方指标、按场景调优预算、组合外部生态，并作出有依据的选型判断。本章以基准复现、调优清单、生态搭配与安全实践收束全教程。

14.1 基准测试复现

官方声称的指标（LongMemEval-S，ICLR 2025 基准的 500 题子集）为：R@5 95.2%、R@10 98.6%、MRR 88.2%，纯 BM25 回退为 86.2% 与 71.5%。这些数字可本地复现：仓库 `eval/README.md` 提供可插拔适配器的评测框架（eval harness），LongMemEval-S 公开集与自建的 `coding-agent-life-v1` 15 会话语料并排评测，grep、纯向量与 agentmemory 三种适配器得分同表输出，NDJSON 格式落盘，评测报告发布在 `docs/benchmarks/`。自行复现时注意官方的诚实声明：对比表中竞品数字多为其自报口径或不同数据集的结果，仅供量级参考。

token 节省口径：对比把全部上下文粘贴进窗口（约 1950 万 token/年，超出窗口不可行）与 LLM 摘要（约 65 万 token/年，约 500 美元），agentmemory 约 17 万 token/年（约 10 美元），配本地嵌入则为 0 成本。

14.2 性能与预算调优清单

目标	手段
降低注入成本	调低 `TOKEN_BUDGET`；保持 `AGENTMEMORY_INJECT_CONTEXT` 按需开启
控制观测体量	设 `AGENTMEMORY_CAPTURE_ALLOW` 白名单或 `AGENTMEMORY_CAPTURE_DENY` 黑名单；调低 `AGENTMEMORY_CAPTURE_OUTPUT_MAX`
避免会话膨胀	关注 `MAX_OBS_PER_SESSION`（默认 500，触发巩固）
大会话摘要	`SUMMARIZE_CHUNK_SIZE`（默认 400）分块 map-reduce，`SUMMARIZE_CHUNK_CONCURRENCY` 控制并行度
加速本地推理	`AGENTMEMORY_LLM_NOTHINK=1` 让本地推理模型跳过隐藏思考链（略降质量）
冷启动向量补齐	大存量历史设 `AGENTMEMORY_VECTOR_BACKFILL=all` 启动全量补嵌

14.3 生态组合

官方推荐与三个项目搭配使用：codegraph（代码图谱索引）、Understand Anything（多智能体构建管线）、Graphify（跨文档、PDF、图像与视频的更广知识图谱）。分工是 agentmemory 记住工作过程，三者为其余上下文层提供能力。搭配配方与问题路由表见仓库 `docs/recipes/pairings.md`。

14.4 安全最佳实践清单

项	建议
密钥管理	共享主机上 `chmod 700 ~/.agentmemory`；远程访问显式设置 `AGENTMEMORY_SECRET`
端口暴露	仅发布 3111；查看器与 iii 控制台保持回环，经 SSH 隧道访问
数据目录	把 `~/.agentmemory/config/` 视为含凭据目录（引擎会在其中持久化展开后的 Redis URL）
Redis	明文连接仅限可信私网；使用权限最小的 ACL 账户
隐私	默认隐私过滤已剥离密钥与 `<private>` 内容；团队共享前仍建议人工复核敏感记忆
遥测	引擎遥测默认关闭，保持关闭除非确有需要

14.5 选型判断与进阶路径

选型判断基于事实而非营销表述：若需求是零依赖、本地优先、多代理共享、自动捕获，agentmemory 的组合在同类中独有；若需要全托管云服务可评估 supermemory、mem0 平台版；若以时序知识图谱为核心需求可评估 Zep/Graphiti；若已在 Oracle 生态则可评估 oracleagentmemory。注意各家指标口径不一，官方对比文档明确标注了哪些数字未经独立复现。

进阶学习路径建议按以下顺序阅读官方文档：Recall（检索排序细节）、Durability（耐久机制）、Governance（治理与审计）、Security（安全模型）、MCP tools（54 工具全表）、REST API（138 端点全表）、Actions（协调层）；二次开发或贡献可读仓库内的 `DESIGN.md` 与 `CONTRIBUTING.md`。版本变更见 `CHANGELOG.md` 与官方文档的 changelog 页。

结语

agentmemory 的核心价值在于把编码代理的隐性经验转化为可检索、可治理、可共享的本地资产。入门的关键动作只有三个：启动服务、接入代理、验证闭环；进阶的关键是理解六类记录与三路检索；精通的标志是能按场景调优预算与捕获范围、可靠地部署与备份，并用官方框架复现指标。全部命令与配置在本文均已注明核实版本与日期，遇到与本文不符的行为时，以官方文档为准。